



# Image Processing

## Lab1

Here's the explanation with comments for each line:

```
import numpy as np # Import the NumPy library, which is used for working with arrays and performing numerical operations.

# Creating arrays of different dimensions:
a = np.array(42) # `a` is a 0-dimensional array (scalar), representing a single value.
b = np.array([1, 2, 3, 4, 5]) # `b` is a 1-dimensional array (vector) containing 5 elements.
c = np.array([[1, 2, 3], [4, 5, 6]]) # `c` is a 2-dimensional array (matrix) with 2 rows and 3 columns.
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
# `d` is a 3-dimensional array (tensor), containing 2 matrices, each with 2 rows and 3 columns.

# Printing the number of dimensions of each array:
print(a.ndim) # Output: 0, because `a` is a 0-dimensional array.
print(b.ndim) # Output: 1, because `b` is a 1-dimensional array.
print(c.ndim) # Output: 2, because `c` is a 2-dimensional array.
print(d.ndim) # Output: 3, because `d` is a 3-dimensional array.
```

### Summary of Dimensions:

- **0-Dimensional (Scalar):** A single value like `42`.
- **1-Dimensional (Vector):** A list of values like `[1, 2, 3, 4, 5]`.
- **2-Dimensional (Matrix):** A table of values like `[[1, 2, 3], [4, 5, 6]]`.
- **3-Dimensional (Tensor):** A stack of matrices like `[[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]]`.

```
import numpy as np

arr = np.array([1, 2, 3, 4], ndmin=5)

print(arr)
print('number of dimensions :', arr.ndim)
```

```
[[[[[1 2 3 4]]]]]
number of dimensions : 5
```

```
my_list = [0,1,2,3,4]
arr = np.array(my_list)
print(arr)
```

```
np.arange(0,10)
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
np.arange(0,10,2)
```

```
array([0, 2, 4, 6, 8])
```

```
np.zeros((5,5))
```

```
"""array([[0., 0., 0., 0., 0.],
          [0., 0., 0., 0., 0.],
          [0., 0., 0., 0., 0.],
          [0., 0., 0., 0., 0.],
          [0., 0., 0., 0., 0.]])"""
```

```
np.ones((2,4))
```

```
"""array([[1., 1., 1., 1.],
          [1., 1., 1., 1.]])"""
```

```
np.random.seed(101)
```

```
# Sets the random seed to 101 to ensure reproducibility of ra
```

```
arr = np.random.randint(0, 100, 10)
```

```
# Generates an array `arr` of 10 random integers between 0 (i
```

```
arr.max()
```

```
# Finds the maximum value in the array `arr`.
```

```
# Example Output: 95 (depends on random seed and values in `a
```

```
arr.min()
```

```
# Finds the minimum value in the array `arr`.
```

```
# Example Output: 8.
```

```
arr.mean()
```

```
# Calculates the mean (average) of all values in `arr`.
```

```
# Example Output: 56.3.
```

```
arr.argmin()
```

```
# Finds the index of the minimum value in `arr`.
```

```
# Example Output: 1 (depends on the array).
```

```
z = arr.reshape(2, 5)
```

```
# Reshapes `arr` into a 2D array with 2 rows and 5 columns.
```

```
# Example Output:
```

```
# [[95 11 81 70 63]
```

```
#  [87 8 62 71 62]]
```

```
z[0, 2]
```

```
# Accesses the element in the 1st row (index 0) and 3rd column
# Example Output: 81.
```

## Explanation and Outputs:

### Reshaping the Array:

```
z = arr.reshape(2, 5)
z
# Reshapes `arr` into a 2D array with 2 rows and 5 columns.
# Example Output:
# [[95 11 81 70 63]
#  [87  8 62 71 62]]
```

### Indexing a Specific Element:

```
z[0, 2]
# Accesses the element in the 1st row (index 0) and 3rd column (index 2) of `z`.
# Example Output: 81
```

### Creating and Indexing a Matrix:

```
mat = np.arange(0, 100).reshape(10, 10)
mat
# Creates a 10x10 matrix with values ranging from 0 to 99.
# Example Output:
# [[ 0  1  2  3  4  5  6  7  8  9]
#  [10 11 12 13 14 15 16 17 18 19]
#  [20 21 22 23 24 25 26 27 28 29]
#  ...
#  [90 91 92 93 94 95 96 97 98 99]]
```

### Accessing a Specific Element:

```
row = 0
col = 1
mat[row, col]
# Accesses the element in the 1st row (index 0) and 2nd column (index 1).
# Example Output: 1
```

### Accessing All Rows for a Specific Column:

```
mat[:, col]
# Accesses all rows for the 2nd column (index 1).
# Example Output:
# [ 1 11 21 31 41 51 61 71 81 91]
```

### Accessing All Columns for a Specific Row:

```
mat[row, :]
# Accesses all columns for the 1st row (index 0).
# Example Output:
# [0 1 2 3 4 5 6 7 8 9]
```

### Slicing a Submatrix:

```
mat[0:3, 0:3]
# Extracts a 3x3 submatrix from the top-left corner.
# Example Output:
# [[ 0  1  2]
#  [10 11 12]
#  [20 21 22]]
```

## Lab2

Here is the **merged, cleaned, and commented** version of the provided code, removing duplicates and redundant parts:

```

# Import necessary libraries
import numpy as np
from matplotlib import image as img # For handling image files
from matplotlib import pyplot as plt # For displaying images
import cv2 # OpenCV library for image processing
from google.colab.patches import cv2_imshow # To display images in Google Colab
from PIL import Image # For handling image files with PIL

# -----
# Loading and displaying an image using Matplotlib
i = img.imread('/content/cat.jpg') # Load the image file
print("Image content values:", i) # Print the pixel values of the image
print("Image type:", type(i)) # Print the type of the image object (numpy.ndarray)
print("Image shape:", i.shape) # Print the dimensions of the image

# Display the image using Matplotlib
plt.title("Cat") # Set the title of the plot
plt.axis('off') # Hide the axes
plt.imshow(i) # Show the image
plt.show()

# Save the image using Matplotlib
plt.imwrite("out1.png", i) # Save the image as 'out1.png'

# -----
# Loading and displaying an image using OpenCV
i2 = cv2.imread('/content/cat.jpg') # Load the image with OpenCV
print("Image type:", type(i2)) # Print the type of the image object (numpy.ndarray)
cv2_imshow(i2) # Display the image using cv2_imshow (Google Colab compatible)

```

```

# Save the image using OpenCV
status = cv2.imwrite('/content/out2.jpg', i2) # Save the image as 'out2.jpg'
print("Image written to file-system:", status) # Confirm if saving was successful

# -----
# Loading and displaying an image using PIL
im = Image.open("/content/cat.jpg") # Open the image file
im.show() # Show the image in the local image viewer (not Colab-compatible)
display(im) # Display the image inline (Colab-compatible)

# -----
# Loading and displaying multiple images using Matplotlib
fig, axs = plt.subplots(2, 2) # Create a grid of 2x2 subplots
images = ['/content/cat.jpg', '/content/cat.jpg', '/content/cat.jpg', '/content/cat.jpg'] # List of images to display

# Loop through each subplot and display the image
for i, ax in enumerate(axs.flat):
    i3 = img.imread(images[i]) # Load each image
    ax.imshow(i3) # Show the image in the subplot
    ax.axis('off') # Hide the axes

plt.show() # Display the grid of images

print(type(axs)) # Print the type of the axs object (numpy.ndarray)
print(axs.shape) # Print the shape of the axs array (2, 2)

# -----
# Exploring NumPy array iteration
a = np.array([[2, 3], [4, 5], [6, 7]]) # Create a 2D NumPy array

```

```
# Iterate through all elements in the array using the `.flat` iterator
for i in a.flat:
    print(i, end=",") # Print each element separated by a comma

# Iterate with both index and value using `enumerate`
for index, value in enumerate(a.flat):
    print(index, "", value) # Print the index and corresponding value
```

---

## Summary of Functionality:

### 1. Image Handling:

- The code demonstrates loading, displaying, and saving images using:
  - **Matplotlib**
  - **OpenCV**
  - **PIL**
- Images are displayed in different styles: single image and grid layout.

### 2. NumPy Array Manipulation:

- Iterates through a 2D NumPy array using `.flat` (flattened iterator).
- Prints elements both with and without their indices.

### 3. Google Colab-Specific Adjustments:

- Uses `cv2.imshow` for displaying images in Colab, as `cv2.imshow` is not supported.

---

## Benefits of Cleaned Code:

- Removes redundancy by avoiding duplicate imports and functions.
- Adds comments for clarity, making it easier to understand each section.
- Organizes the functionality logically (image processing first, NumPy operations later).

# Lab3

```

# Import necessary libraries
import numpy as np
from matplotlib import image as img # For handling image files
from matplotlib import pyplot as plt # For displaying images
import cv2 # OpenCV library for image processing
from google.colab.patches import cv2_imshow # To display images in Google Colab
from PIL import Image # For handling image files with PIL

# -----
# Loading and displaying an image using Matplotlib
i = img.imread('/content/cat.jpg') # Load the image file
print("Image content values:", i) # Print the pixel values of the image
print("Image type:", type(i)) # Print the type of the image object (numpy.ndarray)
print("Image shape:", i.shape) # Print the dimensions of the image

# Display the image using Matplotlib
plt.title("Cat") # Set the title of the plot
plt.axis('off') # Hide the axes
plt.imshow(i) # Show the image
plt.show()

# Save the image using Matplotlib
plt.imsave("out1.png", i) # Save the image as 'out1.png'

# -----
# Loading and displaying an image using OpenCV
i2 = cv2.imread('/content/cat.jpg') # Load the image with OpenCV
print("Image type:", type(i2)) # Print the type of the image object (numpy.ndarray)
cv2_imshow(i2) # Display the image using cv2_imshow (Google Colab compatible)

```

```

# Save the image using OpenCV
status = cv2.imwrite('/content/out2.jpg', i2) # Save the image as 'out2.jpg'
print("Image written to file-system:", status) # Confirm if saving was successful

# -----
# Loading and displaying an image using PIL
im = Image.open("/content/cat.jpg") # Open the image file
im.show() # Show the image in the local image viewer (not Colab-compatible)
display(im) # Display the image inline (Colab-compatible)

# -----
# Loading and displaying multiple images using Matplotlib
fig, axs = plt.subplots(2, 2) # Create a grid of 2x2 subplots
images = ['/content/cat.jpg', '/content/cat.jpg', '/content/cat.jpg', '/content/cat.jpg'] # List of images to display

# Loop through each subplot and display the image
for i, ax in enumerate(axs.flat):
    i3 = img.imread(images[i]) # Load each image
    ax.imshow(i3) # Show the image in the subplot
    ax.axis('off') # Hide the axes

plt.show() # Display the grid of images

print(type(axs)) # Print the type of the axs object (numpy.ndarray)
print(axs.shape) # Print the shape of the axs array (2, 2)

# -----
# Exploring NumPy array iteration
a = np.array([[2, 3], [4, 5], [6, 7]]) # Create a 2D NumPy array

```

```
# Iterate through all elements in the array using the `.flat` iterator
for i in a.flat:
    print(i, end=",") # Print each element separated by a comma

# Iterate with both index and value using `enumerate`
for index, value in enumerate(a.flat):
    print(index, "", value) # Print the index and corresponding value
```

---

## Summary of Functionality:

### 1. Image Handling:

- The code demonstrates loading, displaying, and saving images using:
  - **Matplotlib**
  - **OpenCV**
  - **PIL**
- Images are displayed in different styles: single image and grid layout.

### 2. NumPy Array Manipulation:

- Iterates through a 2D NumPy array using `.flat` (flattened iterator).
- Prints elements both with and without their indices.

### 3. Google Colab-Specific Adjustments:

- Uses `cv2.imshow` for displaying images in Colab, as `cv2.imshow` is not supported.

---

## Benefits of Cleaned Code:

- Removes redundancy by avoiding duplicate imports and functions.
- Adds comments for clarity, making it easier to understand each section.
- Organizes the functionality logically (image processing first, NumPy operations later).

# Lab4

```

# Import necessary libraries
import torchvision.transforms.functional as F # For saturation adjustment
from PIL import Image # For handling images with PIL
import cv2 # OpenCV library for image processing
import matplotlib.pyplot as plt # For displaying images

# -----
# **Adjusting Saturation of an Image using PyTorch**
img = Image.open('panda.jpg') # Open the image using PIL
display(img) # Display the original image

# Adjust the saturation of the image with different factors
img_saturation = F.adjust_saturation(img, 5) # High saturation
display(img_saturation)

img1 = F.adjust_saturation(img, 0) # Black-and-white image
display(img1)

img2 = F.adjust_saturation(img, 0.5) # Reduced saturation
display(img2)

img3 = F.adjust_saturation(img, 1) # Original saturation
display(img3)

# -----
# **Check the file type of an image**
import imghdr
file_type = imghdr.what('/content/2.jpg') # Check the type of the image file
print("File type:", file_type)

# -----
# **Loading and Cropping an Image using OpenCV**
img = cv2.imread('/content/test.webp') # Read the image
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # Convert image from BGR to RGB

```

```

print("Image shape:", img.shape) # Print image dimensions
plt.imshow(img) # Display the original image
plt.show()

# Cropping the image
cropped_image = img[80:280, 150:330] # Crop the image with
slicing
plt.imshow(cropped_image) # Display the cropped image
plt.show()

# -----
# **Image Arithmetic Operations**

# Load two images
img1 = cv2.imread('/content/test-img-1.jpg.webp')
img2 = cv2.imread('/content/test-img-2.jpg.webp')

print("Image 1 shape:", img1.shape)
print("Image 2 shape:", img2.shape)

# Resize the second image to match the dimensions of the fi
rst image
y, x, z = img1.shape
img2_resized = cv2.resize(img2, (x, y))
print("Image 2 shape after resizing:", img2_resized.shape)

# 1. **Image Addition**
img_add = cv2.add(img1, img2_resized) # Add the images
plt.subplot(1, 3, 1), plt.imshow(img1), plt.axis("off"), pl
t.title("Image 1")
plt.subplot(1, 3, 2), plt.imshow(img2_resized), plt.axis("o
ff"), plt.title("Image 2")
plt.subplot(1, 3, 3), plt.imshow(img_add), plt.axis("off"),
plt.title("Image Addition")
plt.show()

# 2. **Image Subtraction**
img_sub = cv2.subtract(img1, img2_resized) # Subtract the

```

```

images
plt.imshow(img_sub), plt.axis("off"), plt.title("Image Subt
raction")
plt.show()

# 3. **Image Multiplication**
img_mul = cv2.multiply(img1, img2_resized) # Multiply the
images
plt.imshow(img_mul), plt.axis("off"), plt.title("Image Mult
iplication")
plt.show()

# 4. **Image Division**
img_div = cv2.divide(img1, img2_resized) # Divide the imag
es
plt.imshow(img_div), plt.axis("off"), plt.title("Image Divi
sion")
plt.show()

```

## Explanation of Key Sections:

### 1. Saturation Adjustment (PyTorch):

- Uses `adjust_saturation()` to change the saturation intensity of an image with factors like `0` (black-and-white), `0.5` (reduced), and `5` (high).

### 2. File Type Check:

- Uses `img_hdr` to determine the type of a given image file.

### 3. Image Cropping (OpenCV):

- Reads and crops the image using slicing.

### 4. Image Arithmetic (OpenCV):

- **Addition:** Combines pixel values of two images.
- **Subtraction:** Subtracts pixel values of one image from another.
- **Multiplication:** Multiplies pixel values of two images.
- **Division:** Divides pixel values of one image by another.

Each operation is visualized using Matplotlib for better understanding.

## Lab5

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load binary images
# `cv2.imread()` reads the image file. The second argument
# `cv2.IMREAD_COLOR` ensures the image is read in color.
img1 = cv2.imread('1bit1.png', cv2.IMREAD_COLOR) # Load the first image
img2 = cv2.imread('2bit2.png', cv2.IMREAD_COLOR) # Load the second image

# Perform Bitwise AND
# `cv2.bitwise_and()` performs a pixel-wise AND operation on the two input images.
# For each pixel, if both corresponding pixel values in the input images are non-zero, the output pixel value is retained.
dest_and = cv2.bitwise_and(img1, img2)

# Display the images using Matplotlib
plt.figure(figsize=(16, 10)) # Set the figure size to 16x10 inches

# Display the first image
# `cv2.cvtColor()` converts the image from BGR (OpenCV default color order) to RGB for correct visualization in Matplotlib.
plt.subplot(1, 3, 1), plt.imshow(cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)), plt.axis("off"), plt.title("Image 1")

# Display the second image
plt.subplot(1, 3, 2), plt.imshow(cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)), plt.axis("off"), plt.title("Image 2")

# Display the result of the bitwise AND operation
```

```
# This shows the combined result where only the overlapping
non-zero regions are visible.
plt.subplot(1, 3, 3), plt.imshow(cv2.cvtColor(dest_and, cv
2.COLOR_BGR2RGB)), plt.axis("off"), plt.title("Bitwise AND
Result")

# Show the complete plot
plt.show()
```

## Explanation of Key Steps:

### 1. Loading the Images:

- `cv2.imread()` reads the binary images.
- The flag `cv2.IMREAD_COLOR` loads the images as 3-channel color images.

### 2. Bitwise AND Operation:

- `cv2.bitwise_and()` is used to compare each pixel in the two images:
  - For each channel (Red, Green, Blue), the AND operation is performed.
  - If both corresponding pixel values are non-zero, the resulting pixel value is retained; otherwise, it is set to 0.

### 3. Visualization:

- OpenCV uses the BGR format, while Matplotlib uses RGB. Hence, `cv2.cvtColor()` is used to convert the color format for proper display.
- `plt.subplot()` creates a grid layout for displaying images side-by-side.
- `plt.axis("off")` hides the axes for better visualization.
- Titles are added using `plt.title()` for clarity.

### 4. Output:

- The resulting image shows the regions common to both input images, highlighted as non-zero pixel values.

## Lab6

Here's the code with detailed comments explaining each line:

```
# Import necessary libraries
import cv2
from matplotlib import pyplot as plt
from PIL import Image, ImageEnhance

# Load the image
imageObj = cv2.imread('/content/red_green.webp')

# Convert the image to RGB format for proper visualization
imageObjrgb = cv2.cvtColor(imageObj, cv2.COLOR_BGR2RGB)

# Display the original image using matplotlib
plt.axis("off") # Turn off axis
plt.title("Original Image") # Title for the image
plt.imshow(imageObjrgb) # Show the image
plt.show() # Display the image

# Get the histogram of each color channel (Blue, Green, and Red)
# cv2.calcHist() calculates the histogram for the specified
channel of the image
blue_color = cv2.calcHist([imageObj], [0], None, [256], [0,
256]) # Blue channel
red_color = cv2.calcHist([imageObj], [1], None, [256], [0,
256]) # Red channel
green_color = cv2.calcHist([imageObj], [2], None, [256],
[0, 256]) # Green channel

# Plot histograms of individual color channels
plt.subplot(3, 1, 1)
plt.title("Histogram of Blue") # Title for Blue histogram
plt.bar(range(0, 256), blue_color.flatten().tolist(), color
='blue') # Plot blue histogram

plt.subplot(3, 1, 2)
plt.title("Histogram of Green") # Title for Green histogra
```

```

m
plt.bar(range(0, 256), green_color.flatten().tolist(), color='green') # Plot green histogram

plt.subplot(3, 1, 3)
plt.title("Histogram of Red") # Title for Red histogram
plt.bar(range(0, 256), red_color.flatten().tolist(), color='red') # Plot red histogram

# Adjust the padding between subplots to ensure clear separation
plt.tight_layout()
plt.show() # Display the histograms of each color channel

# Combined histogram for all RGB channels
plt.title("Histogram of all RGB Colors") # Title for combined histogram
plt.bar(range(0, 256), blue_color.flatten().tolist(), color='blue') # Blue channel
plt.bar(range(0, 256), green_color.flatten().tolist(), color='green') # Green channel
plt.bar(range(0, 256), red_color.flatten().tolist(), color='red') # Red channel
plt.show() # Display the combined RGB histogram

# Alternative way to calculate and display histograms using matplotlib's hist method
# Here, we visualize the Red channel histogram using matplotlib
plt.hist(imageObj[:, :, 2].flatten().tolist(), color="red")
# Flatten the image array and plot red channel
plt.show() # Show the histogram

# Open the image for contrast stretching
image = Image.open("/content/before_contrast_stretching.png")

# Display the image before contrast stretching

```

```

display(image)

# Split the image into Red, Green, and Blue bands for processing
multiBands = image.split()

# Plot histograms of the individual color bands (Red, Green, Blue)
plt.bar(range(0, 256), multiBands[0].histogram(), color='red') # Red band histogram
plt.show()
plt.bar(range(0, 256), multiBands[1].histogram(), color='green') # Green band histogram
plt.show()
plt.bar(range(0, 256), multiBands[2].histogram(), color='blue') # Blue band histogram
plt.show()

# Define a function for contrast stretching for the Red band
def normalizeRed(intensity):
    iI = intensity # Input intensity
    minI = 86 # Minimum intensity in the Red band
    maxI = 230 # Maximum intensity in the Red band
    minO = 0 # Minimum output intensity
    maxO = 255 # Maximum output intensity
    iO = (iI - minI) * ((maxO - minO) / (maxI - minI)) + minO
    # Apply the contrast stretching formula
    return iO

# Define a function for contrast stretching for the Green band
def normalizeGreen(intensity):
    iI = intensity # Input intensity
    minI = 90 # Minimum intensity in the Green band
    maxI = 225 # Maximum intensity in the Green band
    minO = 0 # Minimum output intensity
    maxO = 255 # Maximum output intensity

```

```

    iO = (iI - minI) * (((maxO - minO) / (maxI - minI)) + minO) # Apply the contrast stretching formula
    return iO

# Define a function for contrast stretching for the Blue band
def normalizeBlue(intensity):
    iI = intensity # Input intensity
    minI = 100 # Minimum intensity in the Blue band
    maxI = 210 # Maximum intensity in the Blue band
    minO = 0 # Minimum output intensity
    maxO = 255 # Maximum output intensity
    iO = (iI - minI) * (((maxO - minO) / (maxI - minI)) + minO) # Apply the contrast stretching formula
    return iO

# Apply contrast stretching (point operations) on each color band using the defined functions
normalizedRedBand = multiBands[0].point(normalizeRed) # Apply contrast stretching to Red band
normalizedGreenBand = multiBands[1].point(normalizeGreen)
# Apply contrast stretching to Green band
normalizedBlueBand = multiBands[2].point(normalizeBlue) # Apply contrast stretching to Blue band

# Create a new image by merging the contrast-stretched Red, Green, and Blue bands
normalizedImage = Image.merge("RGB", (normalizedRedBand, normalizedGreenBand, normalizedBlueBand))

# Display the image after contrast stretching
display(normalizedImage)

# Plot the histograms of the contrast-stretched color bands (Red, Green, Blue)
plt.bar(range(0, 256), normalizedRedBand.histogram(), color='red') # Red band histogram
plt.bar(range(0, 256), normalizedGreenBand.histogram(), col

```

```

or='green') # Green band histogram
plt.bar(range(0, 256), normalizedBlueBand.histogram(), color='blue') # Blue band histogram
plt.show()

# Apply contrast enhancement using Pillow's ImageEnhance module
x = img.open("/content/before_contrast_stretching.png") # Open the image
display(x)

# Enhance the contrast by a factor of 3.0
image_enhanced_contrast = ImageEnhance.Contrast(x) # Create an ImageEnhance object for contrast
y = image_enhanced_contrast.enhance(3.0) # Apply the contrast enhancement
display(y) # Display the contrast-enhanced image

# Histogram Equalization: Spread the contrast evenly over the available dynamic range
# Load the image to be equalized
img_before = cv2.imread('/content/2.jpg')
cv2.imshow(img_before) # Display the image before histogram equalization

# Plot the histogram of the original image
plt.hist(img_before.flatten().tolist())
plt.show()

# Equalize the histograms of each color channel (Red, Green, Blue)
eq_b = cv2.equalizeHist(img_before[:, :, 0]) # Equalize the Blue channel
eq_g = cv2.equalizeHist(img_before[:, :, 1]) # Equalize the Green channel
eq_r = cv2.equalizeHist(img_before[:, :, 2]) # Equalize the Red channel

```

```

# Merge the equalized channels back together
merged = cv2.merge([eq_r, eq_g, eq_b])

# Display the image after histogram equalization
cv2_imshow(merged)

# Plot the histogram of the equalized image
plt.hist(merged.flatten().tolist())
plt.show()

# Read and display an image before and after histogram equalization for grayscale images
img1 = cv2.imread('/content/images_4.jpeg', 0) # Load grayscale image
eq_img2 = cv2.equalizeHist(img1) # Perform histogram equalization

cv2_imshow(img1) # Display the original grayscale image
cv2_imshow(eq_img2) # Display the equalized grayscale image

```

## Key Steps Explained:

### 1. Reading and displaying images:

- `cv2.imread()` reads the image.
- `plt.imshow()` displays the image after converting it to the RGB format.

### 2. Calculating histograms:

- `cv2.calcHist()` is used to compute histograms for the Blue, Green, and Red channels of the image.

### 3. Displaying histograms:

- Histograms for each color channel are plotted using `plt.bar()` to show the distribution of pixel intensities.

### 4. Contrast Stretching:

- The function `normalizeRed`, `normalizeGreen`, and `normalizeBlue` apply contrast stretching to each color channel.

- The output image is generated by merging the processed channels.

#### 5. Contrast Enhancement:

- `ImageEnhance.Contrast()` is used to adjust the contrast of the image by a specified enhancement factor.

#### 6. Histogram Equalization:

- `cv2.equalizeHist()` is used to perform histogram equalization on each channel of the image to improve the contrast.

This should give you a good understanding of how histograms, contrast stretching, and histogram

## Lab7 : Sheet

#### 1. Load the two input images:

```
import cv2 # Import OpenCV library for image processing
img1 = cv2.imread('image1.jpg') # Read the first image
from the file system
img2 = cv2.imread('image2.jpg') # Read the second image
from the file system
```

#### 2. Display the two images:

```
from matplotlib import pyplot as plt # Import matplotlib
b for displaying images
plt.imshow(cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)) # Convert BGR (OpenCV default) to RGB and display image1
plt.title("Image 1") # Set the title of the first image window
plt.show() # Display the first image
plt.imshow(cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)) # Convert BGR (OpenCV default) to RGB and display image2
plt.title("Image 2") # Set the title of the second image window
plt.show() # Display the second image
```

**3. Convert the first image with RGB channels into a grayscale image:**

```
img1_gray = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY) # Convert the first image from BGR to grayscale
plt.imshow(img1_gray, cmap='gray') # Display the grayscale image with a grayscale colormap
plt.title("Grayscale Image 1") # Set the title of the grayscale image
plt.show() # Show the grayscale image
```

**4. Apply thresholding to convert the grayscale image to binary image:**

```
_, img1_binary = cv2.threshold(img1_gray, 127, 255, cv2.THRESH_BINARY) # Apply a threshold to convert to binary image
plt.imshow(img1_binary, cmap='gray') # Display the binary image using a grayscale colormap
plt.title("Binary Image 1") # Set the title for the binary image
plt.show() # Show the binary image
```

**5. Resize the second image to be the same size as the first image:**

```
img2_resized = cv2.resize(img2, (img1.shape[1], img1.shape[0])) # Resize image2 to match the dimensions of image1
plt.imshow(cv2.cvtColor(img2_resized, cv2.COLOR_BGR2RGB)) # Convert BGR to RGB for correct display and show resized image2
plt.title("Resized Image 2") # Set the title of the resized image
plt.show() # Display the resized image
```

**6. Apply the addition operation on the two images:**

```
img_add = cv2.add(img1, img2_resized) # Add the two images pixel by pixel
plt.imshow(cv2.cvtColor(img_add, cv2.COLOR_BGR2RGB)) #
```

```
Convert the result to RGB and display the added image
plt.title("Added Image") # Set the title for the added
image
plt.show() # Show the result of the addition
```

**7. Apply the subtraction operation on the two images:**

```
img_sub = cv2.subtract(img1, img2_resized) # Subtract t
he second image from the first image pixel by pixel
plt.imshow(cv2.cvtColor(img_sub, cv2.COLOR_BGR2RGB)) #
Convert to RGB and display the subtracted image
plt.title("Subtracted Image") # Set the title for the s
ubtracted image
plt.show() # Display the result of the subtraction
```

**8. Apply the bitwise AND operation on the two images:**

```
img_and = cv2.bitwise_and(img1, img2_resized) # Perform
bitwise AND operation between the two images
plt.imshow(cv2.cvtColor(img_and, cv2.COLOR_BGR2RGB)) #
Convert the result to RGB and display the AND image
plt.title("Bitwise AND Image") # Set the title for the
bitwise AND image
plt.show() # Display the result of the bitwise AND oper
ation
```

**9. Apply the bitwise NOT operation on the second image:**

```
img_not = cv2.bitwise_not(img2_resized) # Perform bitwi
se NOT operation on the second image
plt.imshow(cv2.cvtColor(img_not, cv2.COLOR_BGR2RGB)) #
Convert to RGB and display the result
plt.title("Bitwise NOT Image") # Set the title for the
bitwise NOT image
plt.show() # Display the result of the bitwise NOT oper
ation
```

**10. Blend the two images using `addWeighted` :**

```

img_blend = cv2.addWeighted(img1, 0.7, img2_resized, 0.3, 0) # Blend the two images with weights of 0.7 and 0.3 respectively
plt.imshow(cv2.cvtColor(img_blend, cv2.COLOR_BGR2RGB))
# Convert to RGB and display the blended image
plt.title("Blended Image") # Set the title for the blended image
plt.show() # Show the blended image

```

**11. Calculate the histogram for each channel of the second image:**

```

blue_hist = cv2.calcHist([img2_resized], [0], None, [256], [0, 256]) # Calculate the histogram for the blue channel (index 0)
green_hist = cv2.calcHist([img2_resized], [1], None, [256], [0, 256]) # Calculate the histogram for the green channel (index 1)
red_hist = cv2.calcHist([img2_resized], [2], None, [256], [0, 256]) # Calculate the histogram for the red channel (index 2)
plt.plot(blue_hist, color='blue') # Plot the blue channel histogram
plt.plot(green_hist, color='green') # Plot the green channel histogram
plt.plot(red_hist, color='red') # Plot the red channel histogram
plt.title("Histogram of Each Channel") # Set the title for the histogram
plt.show() # Display the histogram

```

**12. Display each channel of the second image after applying equalization:**

```

eq_blue = cv2.equalizeHist(img2_resized[:, :, 0]) # Equalize the histogram of the blue channel
eq_green = cv2.equalizeHist(img2_resized[:, :, 1]) # Equalize the histogram of the green channel
eq_red = cv2.equalizeHist(img2_resized[:, :, 2]) # Equalize the histogram of the red channel

```

```
plt.imshow(cv2.merge([eq_blue, eq_green, eq_red])) # Merge the equalized channels and display the result
plt.title("Equalized Image") # Set the title for the equalized image
plt.show() # Show the equalized image
```

**13. Display the second image after applying equalization:**

```
img2_eq = cv2.merge([eq_blue, eq_green, eq_red]) # Merge the equalized channels to form the equalized image
plt.imshow(cv2.cvtColor(img2_eq, cv2.COLOR_BGR2RGB)) # Convert to RGB and display the equalized image
plt.title("Equalized Image 2") # Set the title for the equalized second image
plt.show() # Show the equalized image
```

**14. Load the third input image as grayscale:**

```
img3 = cv2.imread('image3.jpg', cv2.IMREAD_GRAYSCALE) # Read the third image as grayscale
plt.imshow(img3, cmap='gray') # Display the grayscale image using a grayscale colormap
plt.title("Grayscale Image 3") # Set the title for the grayscale third image
plt.show() # Show the grayscale image
```

## Lab9 : Basics of Image Processing

### Key Topics

- **Introduction to Image Processing:**
  - Concepts of digital images.
  - Representation of images in arrays.
- **Reading and Displaying Images:**
  - How to load images using libraries like OpenCV.

- Techniques to visualize images.

## Code Highlights

### 1. Loading Images:

```
import cv2

# Load the image from the specified path
img = cv2.imread('image_path.jpg')

# Display the image in a window
cv2.imshow('Image', img)

# Wait for a key press and close the window
cv2.waitKey(0)
cv2.destroyAllWindows()
```

### 2. Converting to Grayscale:

```
# Convert the loaded image to grayscale
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

### 3. Basic Image Operations:

```
# Access a pixel value at (x, y)
pixel_value = img[50, 100]

# Modify a region of the image
img[100:200, 150:250] = [255, 0, 0] # Change region to
blue
```

### 4. Histogram Analysis:

```
import matplotlib.pyplot as plt

# Calculate the histogram for the grayscale image
hist = cv2.calcHist([gray_img], [0], None, [256], [0, 256])
```

```
# Plot the histogram
plt.plot(hist)
plt.show()
```

## Lab 10: Advanced Image Processing Techniques

### Key Topics

- **Image Filtering:**
  - Smoothing images using techniques like Gaussian and Median filtering.
- **Edge Detection:**
  - Using Canny and Sobel edge detection methods.
- **Geometric Transformations:**
  - Scaling, rotating, and translating images.

### Code Highlights

#### 1. Gaussian Blur:

```
# Apply Gaussian blur to reduce image noise
blurred_img = cv2.GaussianBlur(img, (5, 5), 0)
```

#### 2. Canny Edge Detection:

```
# Detect edges in the image using the Canny method
edges = cv2.Canny(img, 100, 200)
```

#### 3. Affine Transformations:

```
# Get the dimensions of the image
rows, cols, _ = img.shape

# Create a rotation matrix to rotate the image by 45 degrees
M = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 1)
```

```
# Apply the affine transformation
rotated_img = cv2.warpAffine(img, M, (cols, rows))
```

#### 4. Morphological Operations:

```
# Perform dilation on the image
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5,
5))
dilated_img = cv2.dilate(img, kernel, iterations=1)

# Perform erosion on the image
eroded_img = cv2.erode(img, kernel, iterations=1)
```

## Applications

- Enhancing image quality.
  - Preparing images for feature extraction and machine learning tasks.
- 

## Study Tips

- Focus on understanding how image arrays are manipulated.
- Practice using OpenCV functions for common image processing tasks.
- Experiment with different parameter values in filtering and edge detection to see their effects.

لا تنسونا من دعائكم إن أفدناكم ❤️